Micro processors Systems

Introduction to Assembly Language Programming

Dr. Hesham El Zouka

Running Methodology

Step	Input	Program	output
Edit the program	Keyboard	Editor	myfile.asm
Assemble the program	myfile.asm	TASM	myfile.obj
Link the program	myfile.obj	TLINK	myfile.exe

1. **Edit the Program**
   Tool: Keyboard
   Software: Editor
   File Extension: `.asm`

2. **Assemble the Program**
   Input: `myfile.asm`
   Assembler: TASM
   Output: `myfile.obj`

3. **Link the Program**
   Input: `myfile.obj`
   Linker: TLINK
   Executable Output: `myfile.exe`

Structure of an Assembly Language Program

```
. model small                ; Select a memory model
. stack stack_size           ; Define the stack size
. data
                             ; Variable and array declarations
                             ; Declare variables at this level
. code
main proc
                             ; Write the program main code here
main endp
;Other Procedures
                             ; Always organize your program
                             ; into procedures
end main                     ; To mark the end of the source file
```

Example Structure of an Assembly Program

A typical assembly program includes sections for memory model declaration, stack size, data, and code. Example:

```assembly
.model small        ; Selects a memory model suitable for small programs
.stack 64           ; Defines a stack with 64 bytes

.data               ; Start of data segment
  VAR1 DB 25        ; Decimal
  VAR2 DB 10001001b ; Binary
  VAR3 DB 12h       ; Hexadecimal

.code main proc     ; Start of code segment
  ; Write main code here
main endp

end main            ; Marks the end of the program
```

- **Comments**: Always add comments for clarity.

Key Assembly Directives

1. **Model Directive**

   Defines the memory model (e.g., `small`, `large`). The chosen model determines memory allocation.

2. **Stack Segment** (`.stack`)

   Specifies the stack size, a dedicated area for storing temporary data during program execution.

3. **Data Segment** (`.data`)

   Contains variable declarations and constants.

   Example:

   ```assembly
   .data
     DATA1 DB 25      ; Decimal
     DATA2 DB 10001001b ; Binary
   ```

4. **Code Segment** (`.code`)

   Includes executable instructions and procedure calls.

5. **Title Directive**: Specifies the program's title. This directive is optional and acts like a comment, having no effect on the program's execution.

Common Data Definitions

Data types are declared within the data segment:

- **Byte (DB)**: `DB 6 DUP(FFh)` duplicates and fills 6 bytes with `FFh`.
- **Word (DW)**: Allocates 2 bytes.
- **Double-Word (DD)**: Allocates 4 bytes.
- **Quad-Word (DQ)**: Allocates 8 bytes.
- **Ten Bytes (DT)**: Allocates 10 bytes of storage in memory.

```
DW 253Fh,'HI'          ; allocates two bytes
DD 5C2A57F2h           ;allocates four bytes
DQ "HI"                ;allocates eight bytes
COUNT EQU 25
COUNTER1 DB COUNT
COUNTER2 DB COUNT
```

Example:

```assembly
.data
  COUNTER DB 25        ; Defines a counter
  ARRAY DW 954         ; 2-byte word
```

- You can also define an array of bytes using the **DB** directive, like this:

```assembly
DATA1 DB 12, -33, 4, 16, -5  ; Defines an array of bytes with specified values
```

Memory Offset and Data Initialization Examples

Examples of Data Initialization in the Data Segment

The following examples demonstrate how data is initialized in the data segment, with memory offsets ranging from `0000` to `0018`:

```assembly
Offset  | Value
----------------
0000    | ORG 0000h           ; Set origin to 0000h
0000    | DATA1 DB 25          ; Decimal value
0001    | DATA2 DB 10001001b   ; Binary value
0002    | ORG 0010h           ; Set origin to 0010h
0010    | DATA3 DB 12h         ; Hexadecimal value
0011    | DATA4 DB "2591"      ; String value
0018    | DATA5 DB ?           ; Uninitialized value
```

0000 19
0001 89
0002 12
0010 32 35 39 31
0018

In the previous example, **DATA5** is represented by the values `32`, `35`, `39`, and `31`, which correspond to the ASCII characters for the digits '2', '5', '9', and '1', respectively.

Here's how the ASCII representation of these values is stored at the specified offset address:

```assembly
Offset  | Value
----------------
0010    | DATA5 DB 32, 35, 39, 31  ; ASCII values for '2', '5', '9', '1'
```

Example Program: Adding Two Numbers

This program adds two numbers stored in `DATA1` and `DATA2`, with the result stored in `SUM`.

```assembly
.model small
.stack 64

.data
  DATA1 DB 52h
  DATA2 DB 29h
  SUM DB ?

.code
main proc
  MOV AX, @DATA       ; Initialize data segment
  MOV DS, AX
  MOV AL, DATA1       ; Load DATA1 into AL
  MOV BL, DATA2       ; Load DATA2 into BL
  ADD AL, BL          ; Add AL and BL
  MOV SUM, AL         ; Store result in SUM
  MOV AH, 4Ch         ; Terminate program
  INT 21h
main endp
end main
```

Data Transfer Instructions

Data transfer instructions move data from one location to another. Key instructions include:

1. **MOV**: Transfers data from a source to a destination.
   - **Syntax**: `MOV destination, source`

Mnemonic	Meaning	Format	Operation	Flags affected
MOV	Move	MOV D, S	(s) → (D)	None

   - **Example**:

```assembly
MOV AL, BL    ; Moves the contents of BL into AL
```

Using MOV with Two Memory Operands:

```assembly
MOV [VAR1], [VAR2]  ; Illegal: Both operands are memory locations
```

Destination	Source
Memory	Accumulator
Accumulator	Memory
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Seg reg	Reg 16
Seg reg	Mem 16
Reg 16	Seg reg
Memory	Seg reg

2. **XCHG**: Exchanges the contents of two registers.
   - **Example**: `XCHG AX, BX` swaps the contents of AX and BX.
3. **LEA (Load Effective Address)**: Loads the address of a variable.
   - **Example**: `LEA SI, VAR1` loads the address of `VAR1` into SI.

MOV SI, Offset DATA1 or LEA SI, DATA1

Arithmetic Operations in Assembly Language

Mnemonic	Meaning	Format	Operation	Flags Affected
ADD	Addition	ADD D, S	(S) + (D) → (D) Carry → (CF)	All
ADC	Add with carry	ADC D, S	(S) + (D) + (CF) → (D) Carry → (CF)	All
INC	Increment by one	INC D	(D) + 1 → (D)	All but CY
AAA	ASCII adjust for addition	AAA	**If the sum is >9, AH is incremented by 1**	AF,CF
DAA	Decimal adjust for addition	DAA	Adjusts AL for decimal Packed BCD	All

Example: A program designed to add two multi-word numbers.

```
. MODEL SMALL
. STACK 64
.DATA
 DATA1 DQ 548F9963CE7h      ; allocate 8 bytes
ORG 0010h
DATA2 DQ 3FCD4FA23B8Dh; allocate 8 bytes
ORG 0020h
DATA3 DQ   ?
. CODE
 MAIN PROC FAR
 MOV AX, @DATA                          ; receive the starting address for DATA
 MOV DS, AX
 CLC                          ; clear carry
 MOV  SI,  OFFSET  DATA1 ;  LEA  for  DATA1
 MOV  DI,OFFSET  DATA2  ;  LEA  for  DATA2
 MOV  BX,OFFSET  DATA3  ;  LEA  for  DATA3
 MOV CX,04h
 BACK: MOV AX, [SI]
 ADC AX, [DI]                 ; add with carry to AX
 MOV [BX], AX
 ADD   SI,2h
 ADD  DI,2h
 ADD BX,2h
 LOOP BACK                    ; decrement CX automatically until zero
 MOV AH,4Ch
 INT 21h; halt
 MAIN ENDP
 END MAIN
```

Brief Description of the AAA Instruction

The **AAA** (ASCII Adjust AX After Addition) instruction adjusts the result in the AL register after the addition of two ASCII decimal numbers. It ensures that the sum is a valid ASCII representation when the result exceeds 9.

Brief Example

emu8086 - assembler and microprocessor emulator

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
.model small
.data
.code
 main proc

    mov ah, 0
    mov al, '8'      AX = 0038h   ASCII Code (8)
    add al, '2'      AX = 006Ah
    aaa              Ax = 0100h
    or ax, 3030h     AX = 3130h = '10'

 endp

 end main
```

Ex. 2 ASCII CODE 0-9 = 30h – 39h

MOV AX, 38H ;(ASCII code for number 8)

ADD AL, 39H ;(ASCII code for number 9)

AH = 01, AL = 07

AAA; used for addition

ADD AX, 3030H; answer to ASCII → AX = 3137

Brief Description of the DAA Instruction

The **DAA** (Decimal Adjust AX After Addition) instruction in x86 assembly language is used to adjust the result in the AX register after adding two packed BCD (Binary-Coded Decimal) values. When two BCD numbers are added, the resulting sum might not be a valid BCD representation. The DAA instruction corrects the result in AX to ensure that it represents a valid BCD number by adjusting the lower and upper nibbles (4 bits each) of AX if necessary.

Brief Example

- **DAA is Decimal Adjustment after addition.**
- **DAA works with AL register only. Two conditional adjustments are done by DAA Instruction.**
  - Add 06H with AL, If Lower Nibble > 9 or Auxiliary Carry = 1
  - Add 60H with AL, If Higher Nibble > 9 or Carry = 1

Example:

ADD AL,BL
DAA

				AC = 1		CY =1	1 1
AL = 20H	AL = 56H	AL = 36H	AL = 38H	AL = 60H	AL = 90H	AL = 99H
BL = 40H	BL = 23H	BL = 29H	BL = 29H	BL = 70H	BL = 80H	BL = 99H
AL = 60H	AL = 79H	AL = 5FH	AL = 61H	AL = D0H	AL = 10H	AL = 32H
		06H	06H	60H	60H	06H
		AL = 65H	AL = 67H	AL = 30H	AL = 70H	60H
				Carry = 1	Carry = 1	AL = 98H
						Carry = 1

Ex. 2 AL contains 25 (packed BCD)
BL contains 56 (packed BCD)

ADD AL, BL
DAA

```
  25
  56
+ ----------
  7B  → 81
```

Conclusion

This lecture provides a comprehensive overview of assembly language programming, covering the fundamental structure, directives, and common data definitions. Through step-by-step examples, we explore memory management and data initialization, as well as key operations like data transfer and arithmetic instructions. Specific examples, such as adding two numbers and multi-word addition, illustrate practical applications of assembly instructions. We also delve into specialized instructions, like AAA and DAA, to handle ASCII and decimal adjustments. By the end, this guide equips learners with foundational knowledge to create structured, efficient assembly programs and perform basic arithmetic operations.